

PROJECT REPORT

EXPERIMENT – 10

Project title:

INDUSTRIAL DIESEL GENERATOR MONITORING AND
MAINTENANCE SYSTEM

Submitted by:

Arundepak S – 24MER004

Deepak p – 24MER006

Dharanidharan R S – 24MER007

Dinesh K – 24MER009

Vishnivarthan P – 24MER063

INDUSTRIAL DIESEL GENERATOR MONITORING AND MAINTENANCE SYSTEM

1. Project Overview

This project implements an Industrial Diesel Generator Monitoring and Maintenance system built around an ESP32 microcontroller. The system continuously measures generator output current using an ACS712 current sensor and DC bus/battery voltage using a resistive voltage sensor module, monitors ambient temperature and humidity using a DHT11 sensor, displays live readings on a 16x2 I2C LCD, raises a local audible alarm when readings cross safe operating thresholds or the generator becomes due for service, and publishes the sensor data to the Adafruit IO cloud platform over WiFi for remote monitoring and trend analysis.

The solution is designed as a low-cost, easily reproducible IoT monitoring node suitable for industrial diesel generators, backup power installations, or any unattended genset where electrical parameters and running hours need to be tracked for preventive maintenance.

2. Problem Statement

Diesel generators used in industrial and backup power applications are often monitored manually or not monitored at all, which delays the detection of overload, undervoltage, or overheating conditions that can damage the generator or connected equipment. Facility staff frequently have no automated way to track cumulative running hours, so scheduled service intervals are missed or delayed, increasing the risk of unexpected breakdown. There is a need for a low-cost, always-on system that can measure electrical and environmental conditions continuously, alert on-site personnel immediately, track running hours for maintenance scheduling, and make the data available remotely so that generator health can be tracked and analyzed from anywhere.

3. Proposed Solution

An ESP32-based monitoring node is proposed that:

- Reads output current and DC bus voltage at regular intervals using an ACS712 current sensor and a voltage sensor module.
- Reads ambient temperature and humidity using a DHT11 sensor.
- Displays the live current and voltage readings on a local I2C LCD.
- Triggers a buzzer immediately when current, voltage, temperature, or humidity exceeds pre-configured safe thresholds, or when the generator becomes due for scheduled service.
- Tracks cumulative running hours and persists them in EEPROM so the total survives power loss or reset.
- Connects to WiFi and publishes readings to Adafruit IO feeds at fixed intervals.
- Allows facility staff to view live and historical generator data remotely through the Adafruit IO dashboard.

This approach removes the need for manual checks, provides instant local alerting, automates maintenance-interval tracking, and gives remote visibility into generator conditions.

4. System Architecture

The system architecture consists of three layers:

- Sensing & Actuation Layer: ACS712 current sensor, voltage sensor module, and DHT11 sensor for electrical and environmental data acquisition; buzzer for local alerting; and I2C LCD for on-site display.
- Processing & Connectivity Layer: ESP32 reads sensor data, evaluates threshold conditions, accumulates and stores running hours in EEPROM, drives the display and buzzer, and manages the WiFi/Adafruit IO connection.
- Cloud Layer: Adafruit IO receives published readings on individual feeds (current, voltage, temperature, humidity, running hours, alarm status), stores them, and makes them available for visualization and trend charting.

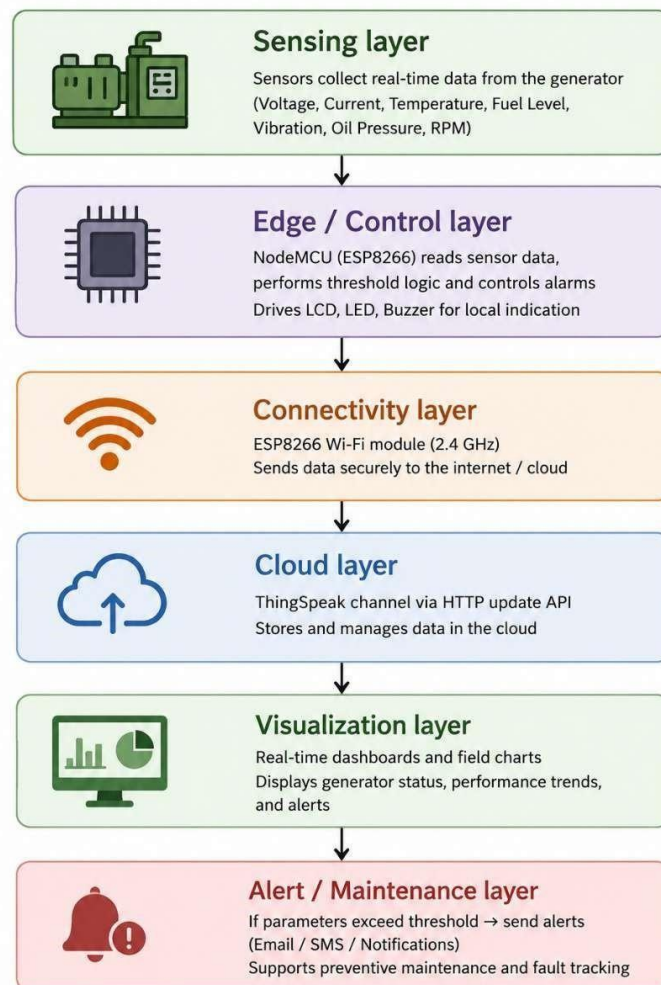


Figure 1: system architecture

Data flow: Sensors → ESP32 (threshold check, running-hours update) → LCD/Buzzer (local feedback) in parallel with ESP32 → WiFi → Adafruit IO Feeds → Remote dashboard

5. Circuit Table

Bill of components used to wire the circuit:

Component	Quantity
ESP32 Dev Module	1
ACS712 Current Sensor	1
0–25V DC Voltage Sensor Module	1
DHT11 Sensor	1
16x2 I2C LCD Display	1
Buzzer Module	1
Breadboard	1
Jumper Wires	1 Set

6. Circuit Design

The DHT11 sensor's data pin, the ACS712 current sensor output, and the voltage sensor output all connect to GPIO/ADC pins on the ESP32, while the I2C LCD connects to the ESP32's SDA and SCL lines. The buzzer is driven from a digital output pin, and all modules share power and ground on the breadboard. The wiring layout is shown below.

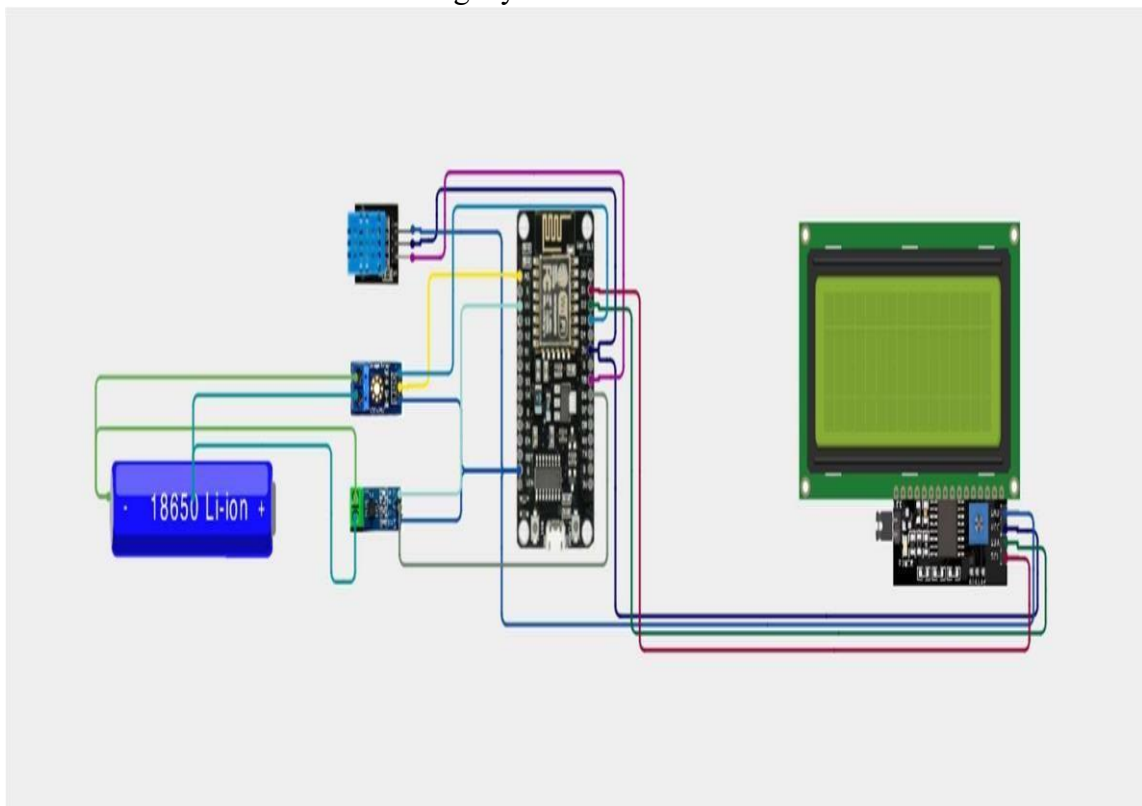


Figure 2: Circuit Design – Sensor and I2C LCD Wiring with ESP32 Dev Module

7. Algorithm

- Step 1: Initialize serial communication, buzzer pin, ADC resolution, I2C bus, DHT11 sensor, and I2C LCD.
- Step 2: Load the previously saved running hours from EEPROM.
- Step 3: Connect to the configured WiFi network and then to Adafruit IO, showing connection status on the LCD.
- Step 4: In the main loop, keep the Adafruit IO connection alive by calling `io.run()`.
- Step 5: Read current and voltage using averaged ADC sampling on the current and voltage sensor pins.
- Step 6: Every 2 seconds, read temperature and humidity from the DHT11 sensor.
- Step 7: Determine generator running state — current above 0.5 A is treated as generator ON — and accumulate running hours accordingly, saving to EEPROM every 60 seconds.
- Step 8: Evaluate alarm conditions — overcurrent, over/under-voltage, overtemperature, over-humidity, or due-for-service.
- Step 9: Update the LCD with the current readings and drive the buzzer HIGH if any alarm condition is active, otherwise keep it LOW.
- Step 10: Print the readings and status to the serial monitor for debugging.
- Step 11: Every publish interval (15 seconds), publish current, voltage, temperature, humidity, running hours, and alarm flag to the Adafruit IO feeds.
- Step 12: Wait briefly and repeat from Step 4.

8. Coding

```
/*  
  
Industrial Diesel Generator Monitoring & Maintenance System  
  
Board: ESP32  
  
Sensors: ACS712 (current), 0-25V DC Voltage Sensor, DHT11  
  
Output: 16x2 I2C LCD, Buzzer  
  
Cloud: Adafruit IO (WiFi)  
  
*/  
  
#include <Wire.h>  
#include <LiquidCrystal_I2C.h>  
#include <DHT.h>  
#include "AdafruitIO_WiFi.h"  
  
// ----- WIFI + ADAFRUIT IO CONFIG -----  
  
#define WIFI_SSID      "test"  
#define WIFI_PASSWORD "test1234"  
#define IO_USERNAME    "dharanidharan_97969"  
#define IO_KEY          "aio_tbXF62lqhm7SXUJV8ryomFcvgvi"      AdafruitIO_WiFi  
io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASSWORD);  
  
// Feeds (must match feed names created on io.adafruit.com)  
AdafruitIO_Feed *currentFeed      = io.feed("current");  
AdafruitIO_Feed *voltageFeed      = io.feed("voltage");  
AdafruitIO_Feed *temperatureFeed =  
io.feed("temperature"); AdafruitIO_Feed *humidityFeed  
= io.feed("humidity");
```

```

AdafruitIO_Feed *hoursFeed      =
io.feed("runninghours"); AdafruitIO_Feed *alarmFeed
=
io.feed("alarm-status");

// ----- PIN CONFIG -----
#define DHTPIN      4
#define DHTTYPE DHT11
#define BUZZER_PIN 25

#define CURRENT_ADC_PIN 34
#define VOLTAGE_ADC_PIN 35

// ----- LCD CONFIG -----
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ----- CALIBRATION -----
#define ADC_MAX_COUNTS
4095.0
#define ADC_VREF      3.3

const float ACS712_SENSITIVITY = 0.066;
const float ACS712_VREF      = 2.500;

const float SENSOR_ONBOARD_RATIO =
5.0; const float VOLTAGE_DIVIDER_RATIO
= 1.0;

// ----- ALARM THRESHOLDS ----- const
float CURRENT_MAX =

```

```

25.0; const float VOLTAGE_MIN =
11.5; const float VOLTAGE_MAX =
14.5; const float TEMP_MAX
           = 60.0;

const float HUMIDITY_MAX =
80.0;

const unsigned long SERVICE_INTERVAL_HRS = 250;


const unsigned long UPLOAD_INTERVAL_MS = 15000; // Adafruit IO free tier ~ 1
update/15s recommended


// ----- EEPROM -----
#include <EEPROM.h>
#define EEPROM_SIZE 16
#define EEPROM_ADDR_HOURS 0


// ----- OBJECTS -----

DHT dht(DHTPIN, DHTTYPE);


// ----- STATE -----

float runningHours = 0.0; unsigned long
lastHourUpdateMillis = 0; unsigned long
lastUploadMillis = 0; unsigned long
lastDHTReadMillis = 0; unsigned long
lastScreenSwitchMillis = 0; int
screenPage = 0; float lastTemp = 0,
lastHum = 0; bool generatorRunning =
false;


// ----- SETUP -----

void setup() { Serial.begin(115200);

```

```
pinMode(BUZZER_PIN, OUTPUT);
digitalWrite(BUZZER_PIN, LOW);

analogReadResolution(12);
analogSetAttenuation(ADC_11db);

Wire.begin(21, 22); dht.begin();

lcd.init();    lcd.backlight();
lcd.setCursor(0, 0);
lcd.print("Genset Monitor");
lcd.setCursor(0, 1);
lcd.print("Connecting IO...");

EEPROM.begin(EEPROM_SIZE);
EEPROM.get(EEPROM_ADDR_HOURS, runningHours);
if (isnan(runningHours) || runningHours < 0) runningHours = 0;

// Connect to Adafruit IO
Serial.print("Connecting to Adafruit IO");
io.connect(); while (io.status() <
AIO_CONNECTED) {
    Serial.print(".");
    delay(500);
}
Serial.println();
Serial.println(io.statusText());

lcd.clear();
lastHourUpdateMillis =
```

```
    millis(); lastScreenSwitchMillis =  
    millis();  
}
```

```
// ----- MAIN LOOP -----
```

```
void loop() {  
    io.run(); // REQUIRED - keeps Adafruit IO connection alive
```

```
  
    float current = readCurrentAmps(); float  
    voltage = readVoltage();
```

```
  
    if (millis() - lastDHTReadMillis > 2000) {  
        lastDHTReadMillis = millis(); float h =  
        dht.readHumidity();  
        float t =  
        dht.readTemperature(); if  
        (!isnan(h)) lastHum = h; if  
        (!isnan(t)) lastTemp = t;  
    }
```

```
  
    generatorRunning = (current > 0.5); updateRunningHours();
```

```
  
    bool alarm = checkAlarms(current, voltage, lastTemp, lastHum);  
    digitalWrite(BUZZER_PIN, alarm ? HIGH : LOW);
```

```
  
    if (millis() - lastScreenSwitchMillis > 3000)  
    { lastScreenSwitchMillis = millis();  
      screenPage = (screenPage + 1) % 2;  
    }
```

```
    updateLCD(current, voltage, lastTemp, lastHum, alarm);
```

```
Serial.print("I:"); Serial.print(current, 2);  
Serial.print(" V:"); Serial.print(voltage, 2);  
Serial.print(" T:"); Serial.print(lastTemp, 1);  
Serial.print(" H:"); Serial.print(lastHum, 1);  
Serial.print(" Hrs:"); Serial.print(runningHours, 2);  
Serial.println(alarm ? " ALARM!" : " OK");
```

```
if (millis() - lastUploadMillis > UPLOAD_INTERVAL_MS) {  
    lastUploadMillis = millis(); uploadToAdafruitIO(current, voltage,  
    lastTemp, lastHum, alarm)  
}
```

```
    delay(500);  
}
```

```
// ----- SENSOR READS -----
```

```
float readPinVoltage(int pin, int samples = 20) {  
    long sum = 0; for (int i = 0; i < samples; i++) {  
        sum += analogRead(pin);  
        delayMicroseconds(200);  
    }    float    avgCounts    =    sum    /  
    (float)samples;  
    return (avgCounts / ADC_MAX_COUNTS) * ADC_VREF;  
}
```

```
float    readCurrentAmps()    {    float    vPin    =  
    readPinVoltage(CURRENT_ADC_PIN); return (vPin  
    - ACS712_VREF) /  
    ACS712_SENSITIVITY;
```

```
}
```

```
float    readVoltage()    {    float    vPin    =  
    readPinVoltage(VOLTAGE_ADC_PIN);    float  
    vSensorOut = vPin /  
    VOLTAGE_DIVIDER_RATIO; return vSensorOut *  
    SENSOR_ONBOARD_RATIO;
```

```
// ----- MAINTENANCE LOGIC ----- void
```

```
updateRunningHours()  
{ if (generatorRunning) { unsigned long now =  
    millis();    float    elapsedHours    =    (now    -  
    lastHourUpdateMillis) / 3600000.0; runningHours  
    += elapsedHours;  
    lastHourUpdateMillis = now;
```

```
static unsigned long lastSave =  
0; if (now - lastSave > 60000) {  
    lastSave = now;  
    EEPROM.put(EEPROM_ADDR_HOURS, runningHours);  
    EEPROM.commit();  
}
```

```
} else { lastHourUpdateMillis =  
    millis();  
}
```

```
}
```

```
// ----- ALARM LOGIC -----
```

```
bool checkAlarms(float current, float voltage, float temp, float hum) { bool alarm = false;  
    if (current > CURRENT_MAX) alarm = true; if (voltage > 0 && (voltage <
```

```

    VOLTAGE_MIN || voltage > VOLTAGE_MAX)) alarm = true; if (temp > TEMP_MAX)
    alarm = true; if (hum > HUMIDITY_MAX) alarm = true;

    if (runningHours >= SERVICE_INTERVAL_HRS && fmod(runningHours,
SERVICE_INTERVAL_HRS) < 5) alarm = true

    return alarm;
}

// ----- DISPLAY -----

void updateLCD(float current, float voltage, float temp, float hum, bool alarm) {
    lcd.setCursor(0, 0);
    lcd.print("I:"); lcd.print(current, 2); lcd.print("A "); // trailing spaces clear old chars

    lcd.setCursor(0, 1); lcd.print("V:"); lcd.print(voltage,
2); lcd.print("V ");
}

// ----- ADAFRUIT IO UPLOAD ----- void
uploadToAdafruitIO(float current, float voltage, float temp, float hum, bool
alarm) { currentFeed->save(current); voltageFeed->save(voltage);
temperatureFeed->save(temp); humidityFeed->save(hum); hoursFeed-
>save(runningHours);          alarmFeed-
>save(alarm ? 1 : 0);

    Serial.println("Uploaded to Adafruit IO");
}

```

9. System Working

On power-up, the ESP32 initializes its peripherals and connects to WiFi and then to Adafruit IO, showing connection progress on the LCD. Once connected, the system enters its main loop: it continuously samples current and voltage through averaged ADC readings, and every 2 seconds reads temperature and humidity from the DHT11 sensor. As long as the sensed current exceeds 0.5 A, the generator is considered running and its operating hours are accumulated in real time, with the total saved to EEPROM every minute so it is never lost on reset or power loss.

If the current, voltage, temperature, or humidity moves outside the configured safe band, or the running hours reach the 250-hour service interval, the buzzer is switched on immediately to give an audible local warning; it switches off again once conditions return to normal. The LCD continuously displays the live current and voltage readings. In parallel, every 15 seconds the current, voltage, temperature, humidity, running hours, and alarm flag are packaged and published to the respective Adafruit IO feeds, so the same data shown locally on the LCD becomes visible on the remote Adafruit IO dashboard for live monitoring and trend analysis.

The Arduino IDE screenshots below show the alarm logic, LCD update function, and Adafruit IO upload function alongside live serial monitor output confirming this behaviour.

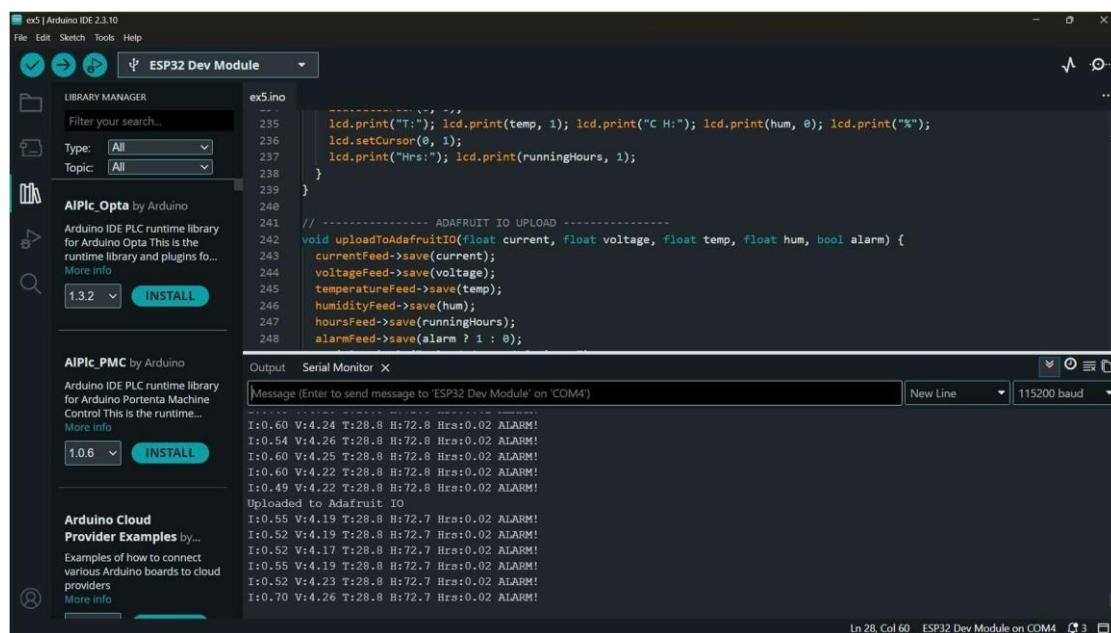


Figure 3: Arduino IDE – Alarm Logic and LCD Update Functions

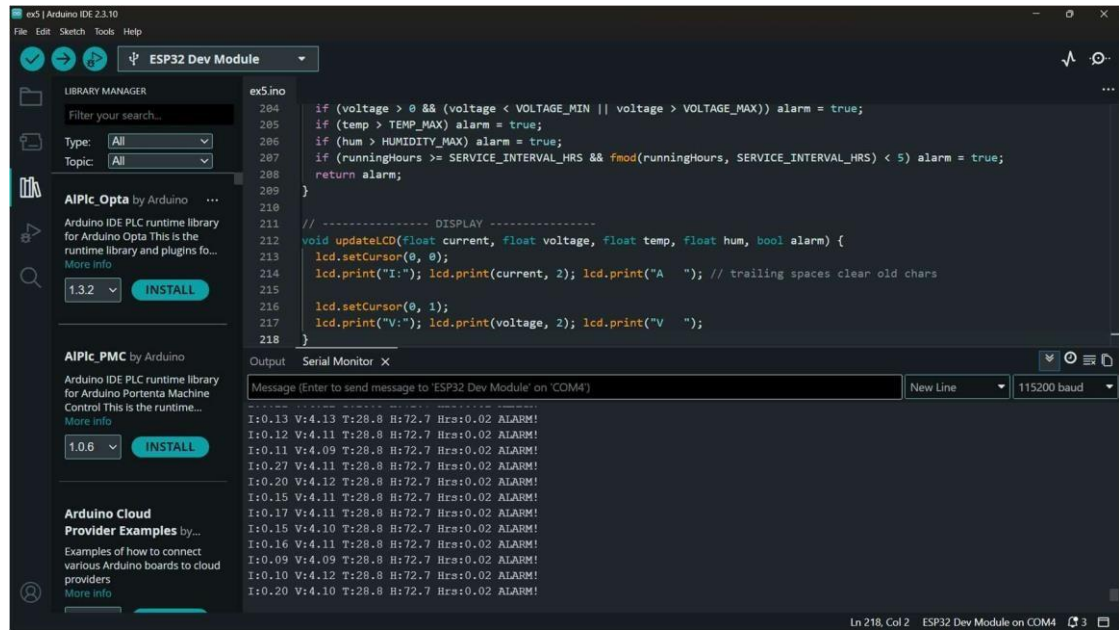


Figure 4: Arduino IDE – Adafruit IO Upload Function and Serial Monitor Output

10. Bill of Materials

S.No	Component	Quantity
1	ESP32 Dev Module (NodeMCU-32S)	1
2	ACS712 Current Sensor Module	1
3	0–25V DC Voltage Sensor Module	1
4	DHT11 Temperature & Humidity Sensor	1
5	16x2 I2C LCD Display	1
6	Active Buzzer Module	1
7	18650 Li-ion Cell (prototype supply)	1
8	Breadboard	1
9	Jumper Wires	1 Set

11. Prototype Implementation

The prototype was assembled on a breadboard with the ACS712 current sensor, voltage sensor module, DHT11 sensor, I2C LCD, and buzzer wired to the ESP32 Dev Module as per the circuit design, powered by an 18650 Li-ion cell for benchtop testing. Firmware was uploaded using the Arduino IDE, and the device was verified to connect to the local WiFi network and Adafruit IO on power-up, after which live readings and status were confirmed on both the LCD and the serial monitor.

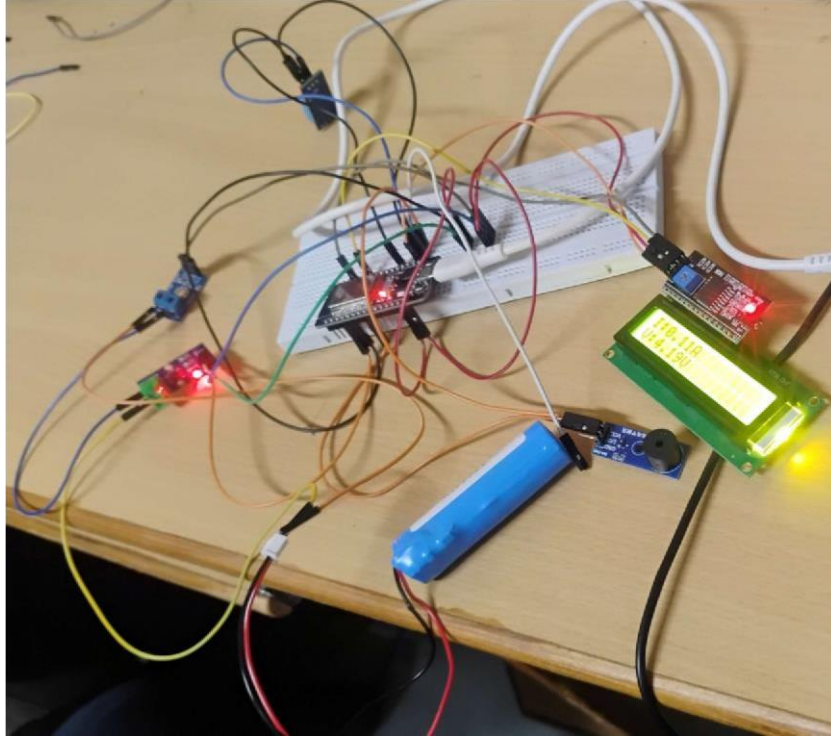


Figure 5: Prototype Hardware – Assembled Circuit with Live LCD Readout

12. Results & Performance Analysis

The device was registered on Adafruit IO with feeds for current, voltage, temperature, humidity, running hours, and alarm status, confirming successful cloud connectivity. The live dashboard displays gauges for each parameter along with a running-hours trend chart. The figures below show the dashboard at two different points during testing.

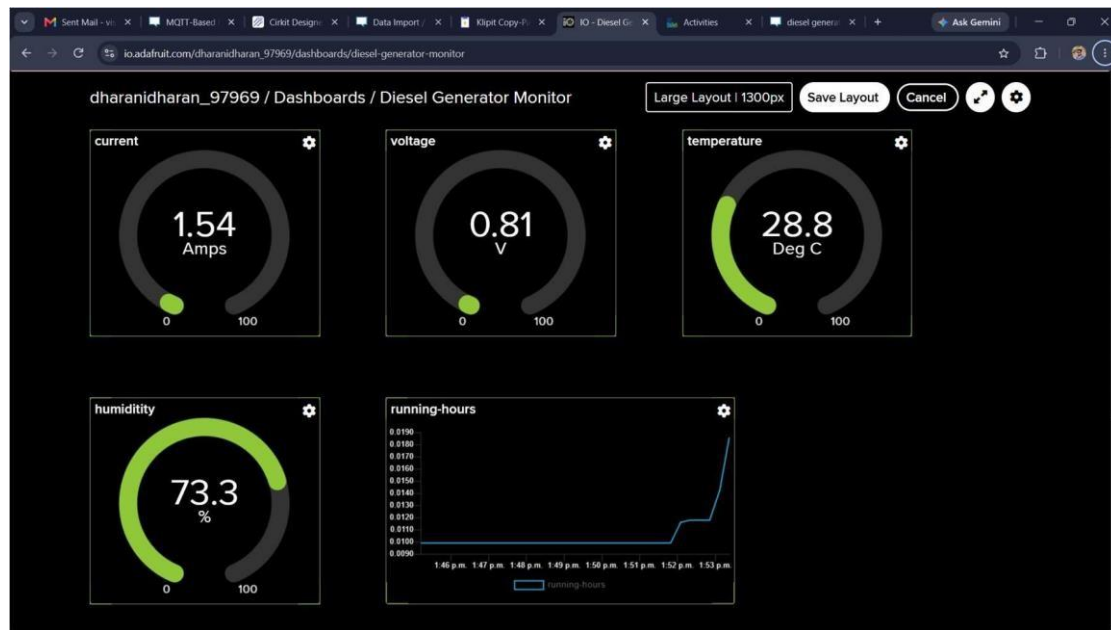


Figure 6: Adafruit IO Dashboard – Initial Live Readings (I: 1.54 A, V: 0.81 V, T: 28.8°C, H: 73.3%)

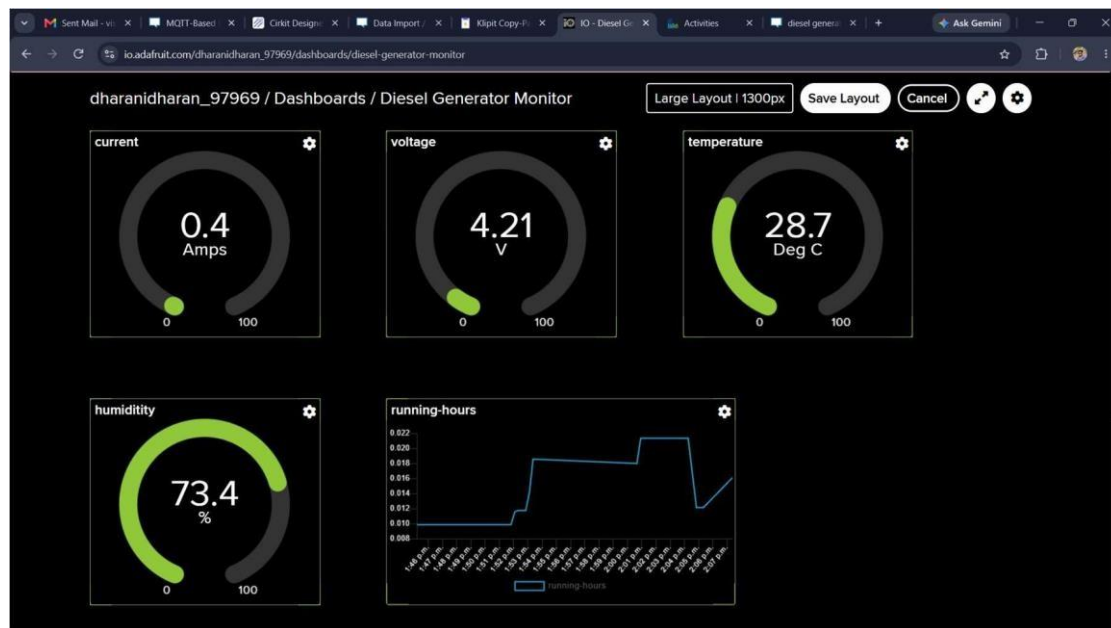


Figure 7: Adafruit IO Dashboard – Running-Hours Trend Update (I: 0.4 A, V: 4.21 V, T: 28.7°C, H: 73.4%)

The system was tested over multiple operating cycles with the following observations:

- Sensor readings updated reliably and were correctly reflected on both the LCD and the Adafruit IO dashboard.
- The running-hours chart showed a continuously increasing trend while current remained above the 0.5 A running threshold, confirming accurate operating-time accumulation and EEPROM persistence.

- The buzzer activated immediately when voltage readings fell outside the configured 11.5–14.5 V band, and deactivated once conditions returned to normal.
- Adafruit IO publishes succeeded consistently at the configured 15-second interval, with no dropped uploads observed in the serial monitor log.
- Temperature and humidity readings remained stable and consistent with ambient lab conditions, confirming reliable DHT11 sensor integration.